Intego X9: When your macOS antivirus becomes your enemy

Posted Tue 10 February 2026
Author Mathieu Farrell
Category Vulnerability
Tags 2026, pentest, Intego, macOS, vulnerability, antivirus, CVE-2026-26224, CVE-2026-26225

This blog post dives into the most common classes of macOS Local Privilege Escalation vulnerabilities, from time-of-check to time-of-use (TOCTOU) Race Conditions and insecure XPC communications to a range of implementation and configuration oversights. We will explore how attackers can exploit these weaknesses to escalate privileges, and highlight real-world examples to illustrate recurring patterns.

Author's note

This article is part of a series of blog posts dedicated to identify vulnerabilities in third-party macOS applications. The goal is to document real-world flaws and explain the techniques used to discover and exploit them. Other examples of this series include our prior posts about ControlPlane, CCleaner, and Microsoft Teams:

- ControlPlane Local Privilege Escalation Vulnerability on macOS
- CCleaner Local Privilege Escalation Vulnerability on macOS
- Exploiting Microsoft Teams on macOS during a Purple Team engagement

As my latest target is Intego X9, this topic will be covered over three blog posts, due to the number of issues discovered. Each part will focus on different aspects of the findings, including technical details, exploitation techniques, and the overall security implications for macOS users relying on Intego products.

Introduction

In modern macOS environments, threat surface extends far beyond traditional viruses. They encompass network intrusions, data corruption, performance degradation and user misuse. Intego is a security software vendor with a product portfolio that spans from antivirus, firewall, parental control, system-cleaning to optimization tools, backup solutions, and privacy/VPN utilities.

In this article I present the results of research outlining identification and exploitation of several vulnerabilities that enabled compromise of **all Intego applications** on macOS.

The vendor offers a downloadable bundle (the installation method we used).

> Testing shows that the vulnerabilities are exploitable whether the applications are installed individually or through the full bundle, meaning that any user running one or more of these products is at risk.

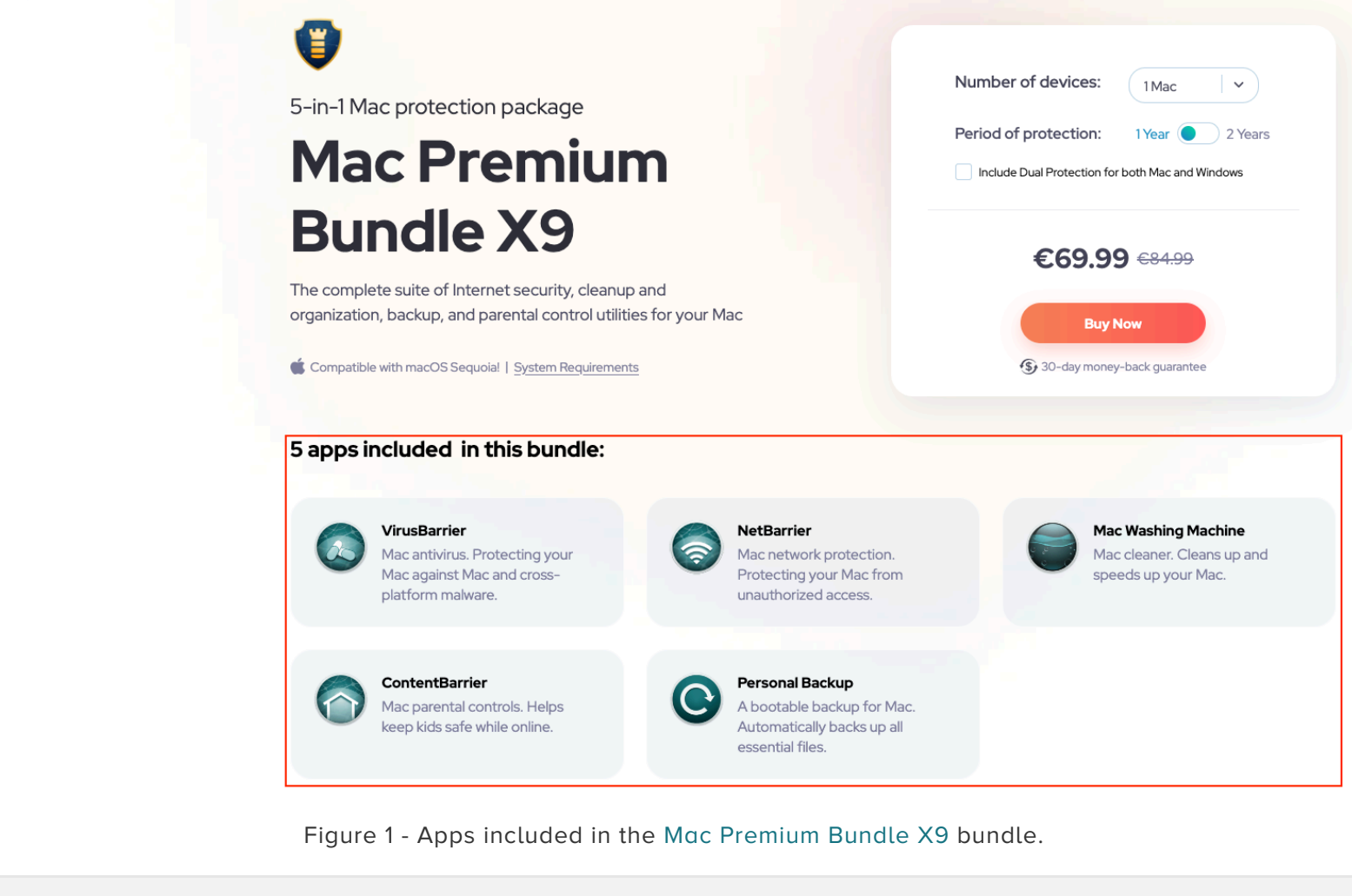
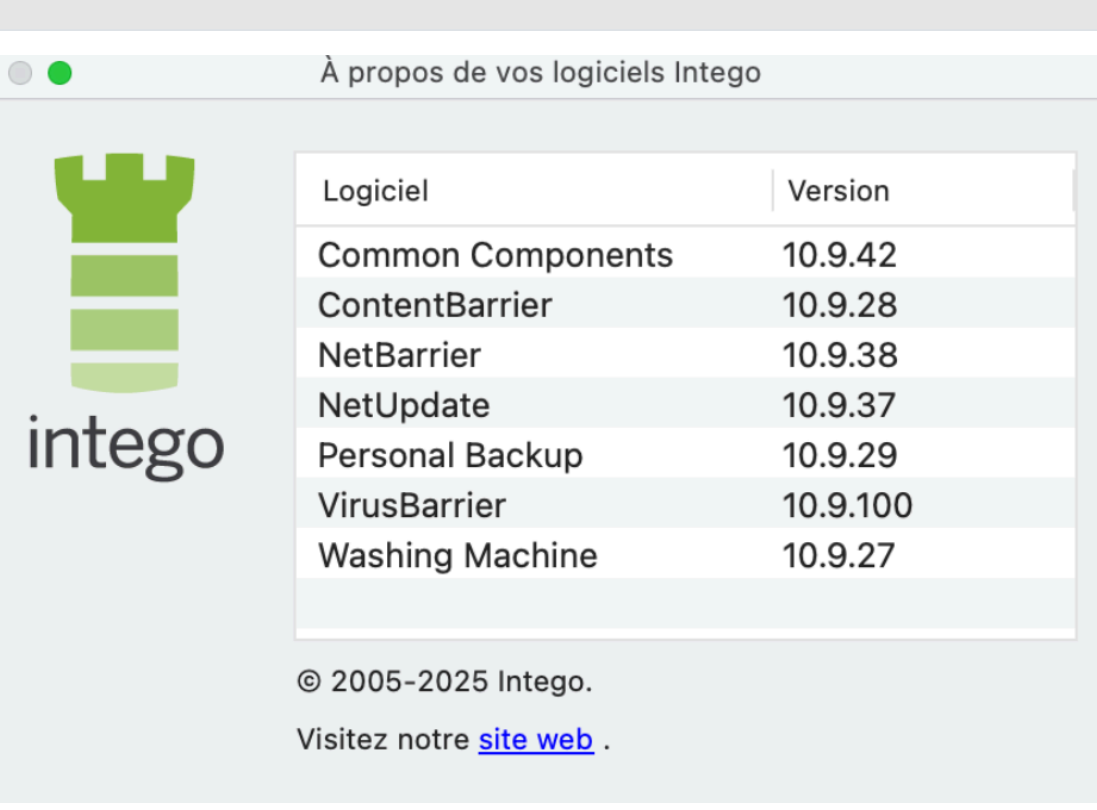

www.intego.com/en/mac-protection-bundle
5-in-1 Mac protection package
Mac Premium Bundle X9
The complete suite of Internet security, cleanup and organization, backup, and parental control utilities for your Mac
Compatible with macOS Sequoia | System Requirements
Number of devices: 1 Mac
Period of protection: 1 Year 2 Years
Include Dual Protection for both Mac and Windows
€69.99 €84.99
Buy Now
30-day money-back guarantee
5 apps included in this bundle:
VirusBarrier
Mac antivirus. Protecting your Mac against Mac and cross-platform malware.
NetBarrier
Mac network protection. Protecting your Mac from unauthorized access.
Mac Washing Machine
Mac cleaner. Cleans up and speeds up your Mac.
ContentBarrier
Mac parental controls. Helps keep kids safe while online.
Personal Backup
A bootable backup for Mac. Automatically backs up all essential files.

Figure 1 - Apps included in the Mac Premium Bundle X9 bundle.

> The following descriptions are sourced from the vendor's official website.

Icon	Application	Description
	VirusBarrier	Mac antivirus. Protecting your Mac against Mac and cross-platform malware.
	NetBarrier	Mac network protection. Protecting your Mac from unauthorized access.
	Mac Washing Machine	Mac cleaner. Cleans up and speeds up your Mac.
	ContentBarrier	Mac parental controls. Helps keep kids safe while online.
	Personal Backup	A bootable backup for Mac. Automatically backs up all essential files.



À propos de vos logiciels Intego

Logiciel	Version
Common Components	10.9.42
ContentBarrier	10.9.28
NetBarrier	10.9.38
NetUpdate	10.9.37
Personal Backup	10.9.29
VirusBarrier	10.9.100
Washing Machine	10.9.27

© 2005-2025 Intego.
Visitez notre site web .

Multiple vulnerabilities were discovered and disclosed to the vendor, though only two will be discussed in detail in this blog post, the others will be published later:

- Intego Log Reporter Local Privilege Escalation (as `root`)
- Intego Personal Backup Local Privilege Escalation (as `root`)

Intego Log Reporter Local Privilege Escalation (as `root`)

- CVE identifier:
  - CVE-2026-26224

> The following information was obtained from Intego Support.

Intego Log Reporter is a diagnostic tool bundled with each Intego application that collects detailed system and application logs as well as configuration data to help diagnose issues. When triggered, it builds a report that gathers error messages, application specific log files, user and system metadata, and then compresses all this information into a single archive (.tgz file) that the user can send to Intego Support for analysis.

- Explanation of how the application works, extracted from Intego's support (PDF).

One critical component of the tool is executed with `root` privileges, which allows it to access sensitive areas of the macOS filesystem that standard users cannot reach. The elevated access enables Intego Log Reporter to retrieve system logs, configuration files, and diagnostic information from protected directories, providing deeper visibility for analysis and troubleshooting purposes.

The information collection logic of Intego Log Reporter is implemented as a Bash script located at:

- /Library/Intego/commonservices.bundle/Contents/MacOS/IntegoLogReporter.app/Contents/Resources/idiagnose

The main application executes the script as `root` to gather and read sensitive system data, including logs and Firefox profiles. However, the script writes collected data to /tmp without enforcing any security checks, which introduces a TOCTOU (time-of-check to time-of-use) race condition vulnerability. This oversight allows an unprivileged user to perform a symlink-based TOCTOU attack, allowing the attacker to write to sensitive locations like /etc/sudoers.d/ and achieve local privilege escalation as `root`.

File: /Library/Intego/commonservices.bundle/.../IntegoLogReporter.app/Contents/Resources/idiagnose

```
#!/bin/sh

...

OSVersion=`sysctl -n kern.osrelease | cut -f 1 -d .`

# Compose directory name.
current_date=`date "+%Y.%m.%d_%H-%M-%S"`
temp_directory="/tmp"
output_directory_name="IntegoLog_${current_date}_${USER}"
output_directory_path="${temp_directory}/${output_directory_name}"

# Clean directory.
/bin/rm -rf "${output_directory_path}"
/bin/mkdir "${output_directory_path}"

# Enter directory.
cd "${output_directory_path}"
output_directory_path=`pwd -P`

# macOS
echo "Getting macOS version."
product_name=`sw_vers -productName`
product_version=`sw_vers -productVersion`
build_version=`sw_vers -buildVersion`

/bin/mkdir "${output_directory_path}/ ${product_name} ${product_version} (${build_version})"

...

#############     OS and Browsers notifications     #############
echo "OS and Browsers notifications..."
notifications_directory="${output_directory_path}/os_and_browsers_notifications"
/bin/mkdir "${notifications_directory}"

...

sourcepath="/Users/${USER}/Library/Application Support/Firefox/Profiles"
if [ -d "${sourcepath}" ]; then
    /usr/bin/find "${sourcepath}"/* -prune -type d | while IFS= read -r d; do
        settingsfile="${d}/notificationstore.json"
        if [ -f "${settingsfile}" ]; then
            filename=$(basename "${d}")
            /bin/cp  "${settingsfile}" "${notifications_directory}/Firefox-${filename}-notificationstore.json"
        fi
    done
fi

...
```

We developed an exploit that specifically targets this Race Condition, focusing on the insecure handling of temporary directory creation and file copying. The exploit works by setting up a malicious Firefox profile (even if it is not installed) and a backdoor, which together trigger the privilege escalation process. The full exploit code is available here.

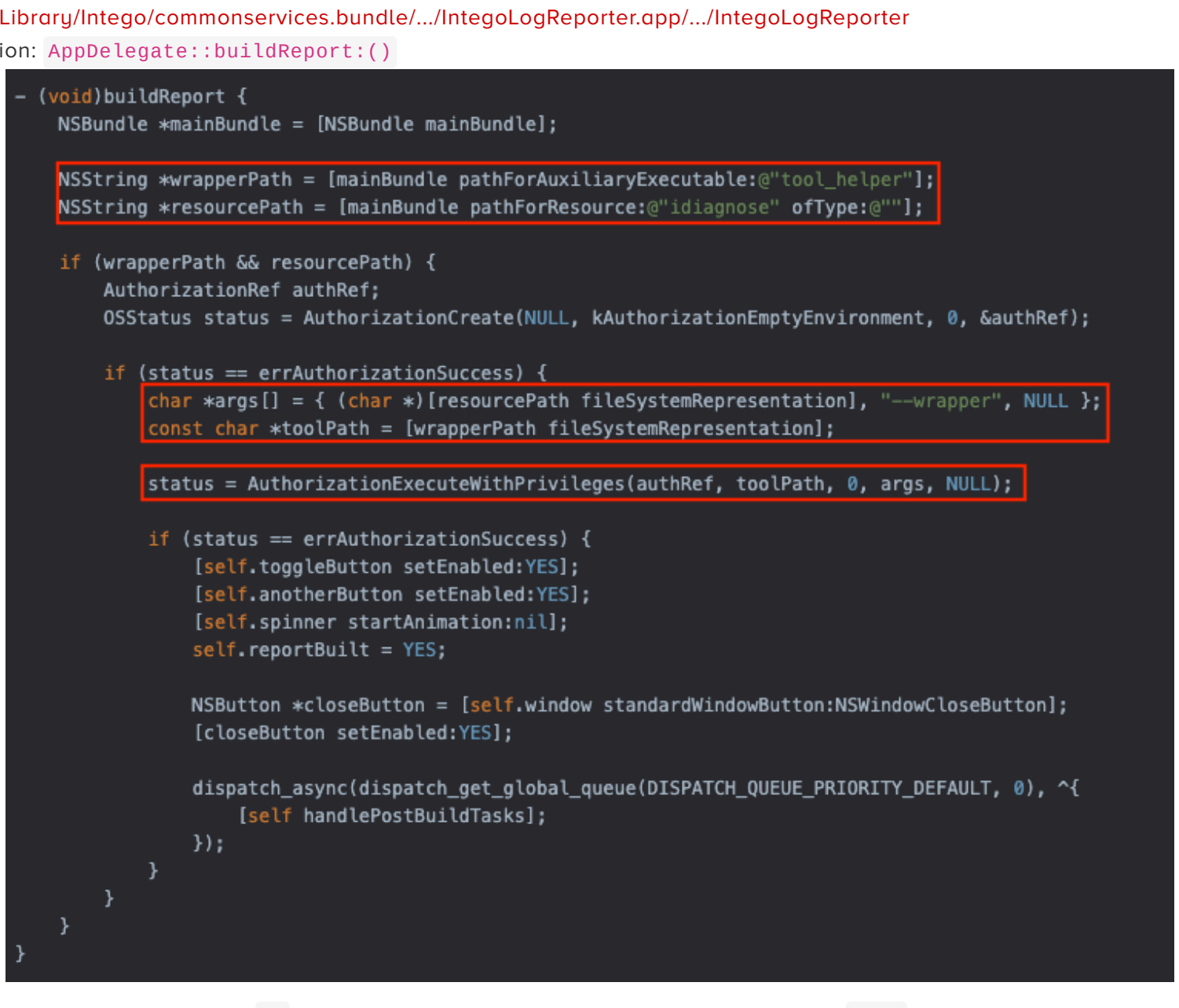
It starts by creating a fake Firefox profile and a notificationstore.json file in the user's library directory. Then, it waits for the idiagnose script to execute. Once detected, the exploit begins creating threads that exploit the Race Condition. These threads create directories and symbolic links in the /tmp directory at the precise moment when the script is running. This timing flaw allows files to be copied into arbitrary target directories (here, /etc/sudoers.d/ was chosen as a target).

After the exploit completes, the script removes all temporary files and directories (including the fake Firefox profile), leaving no trace of the attack.

Root cause analysis

To determine the root cause of the vulnerability, we used a reverse-engineering tool such as Ghidra to inspect and analyze the Objective-C code of the relevant components. The vulnerable script (idiagnose running as `root`) is launched by Intego Log Reporter (/Library/Intego/commonservices.bundle/.../IntegoLogReporter.app/.../IntegoLogReporter), whose equivalent Objective-C code could hypothetically be:

File: /Library/Intego/commonservices.bundle/.../IntegoLogReporter.app/.../IntegoLogReporter
Function: `AppDelegate::buildReport:()`



```
- (void)buildReport {
    NSBundle *mainBundle = [NSBundle mainBundle];

    NSString *wrapperPath = [mainBundle pathForAuxiliaryExecutable:@"tool_helper"];
    NSString *resourcePath = [mainBundle pathForResource:@"idiagnose" ofType:@""];

    if (wrapperPath && resourcePath) {
        AuthorizationRef authRef;
        OSStatus status = AuthorizationCreate(NULL, kAuthorizationEmptyEnvironment, 0, &authRef);

        if (status == errAuthorizationSuccess) {
            char *args[] = { (char *)[resourcePath fileSystemRepresentation], "--wrapper", NULL };
            const char *toolPath = [wrapperPath fileSystemRepresentation];

            status = AuthorizationExecuteWithPrivileges(authRef, toolPath, 0, args, NULL);

            if (status == errAuthorizationSuccess) {
                [self.toggleButton setEnabled:YES];
                [self.anotherButton setEnabled:YES];
                [self.spinner startAnimation:nil];
                self.reportBuilt = YES;

                NSButton *closeButton = [self.window standardWindowButton:NSWindowCloseButton];
                [closeButton setEnabled:YES];

                dispatch_async(dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_DEFAULT, 0), ^{
                    [self handlePostBuildTasks];
                });
            }
        }
    }
}
```

As shown in the output of the `ps` command below, the vulnerable script is running as `root`.

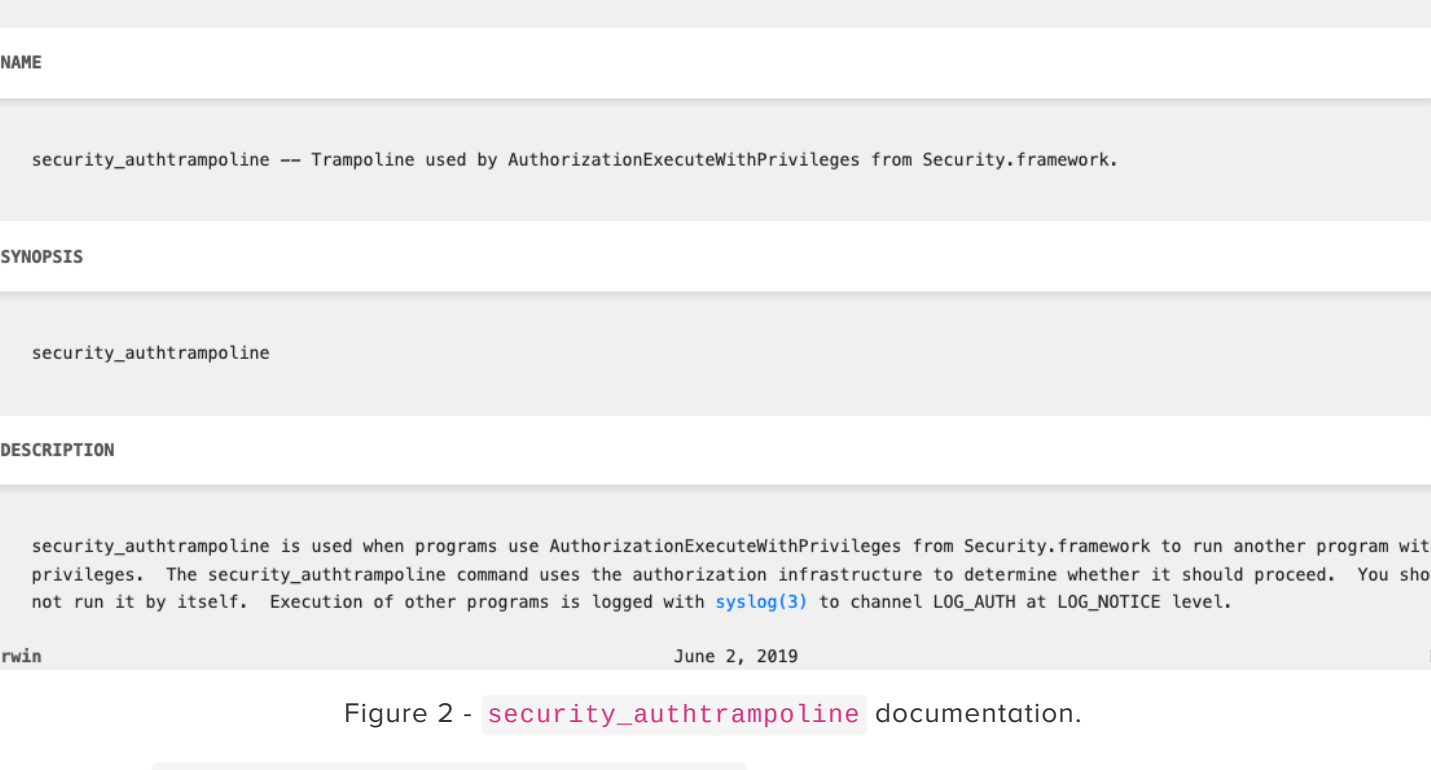
Command:

```
ps aux | grep idiagnose
```

Output:

```
root 662 0,0 0,0 408507968 2832 s000 S+ 7:35 0:00.00 /usr/libexec/security_authtrampoline
/Library/Intego/commonservices.bundle/Contents/MacOS/IntegoLogReporter.app/Contents/MacOS/tool_helper auth 8
/Library/Intego/commonservices.bundle/Contents/MacOS/IntegoLogReporter.app/Contents/Resources/idiagnose --wrapper
```

The command path `/usr/libexec/security_authtrampoline` is designed to invoke system commands requiring elevated permissions (such as `sudo`). The command proceeds to execute the `tool_helper` binary, which resides inside the IntegoLogReporter.app bundle.



```
security_authtrampoline(8)          BSD System Manager's Manual          security_authtrampoli

NAME

     security_authtrampoline -- Trampoline used by AuthorizationExecuteWithPrivileges from Security.framework.

SYNOPSIS

     security_authtrampoline

DESCRIPTION

     security_authtrampoline is used when programs use AuthorizationExecuteWithPrivileges from Security.framework to run another program with
     privileges.  The security_authtrampoline command uses the authorization infrastructure to determine whether it should proceed.  You shou
     not run it by itself.  Execution of other programs is logged with syslog(3) to channel LOG_AUTH at LOG_NOTICE level.

Darwin                                June 2, 2019                                D
```

Figure 2 - `security_authtrampoline` documentation.

To understand how `AuthorizationExecuteWithPrivileges()` works, we invite you to read Patrick Wardle's article Sniffing Authentication References on macOS.

Exploit

Here is the link to the exploit written for Python3:

- exploit.py

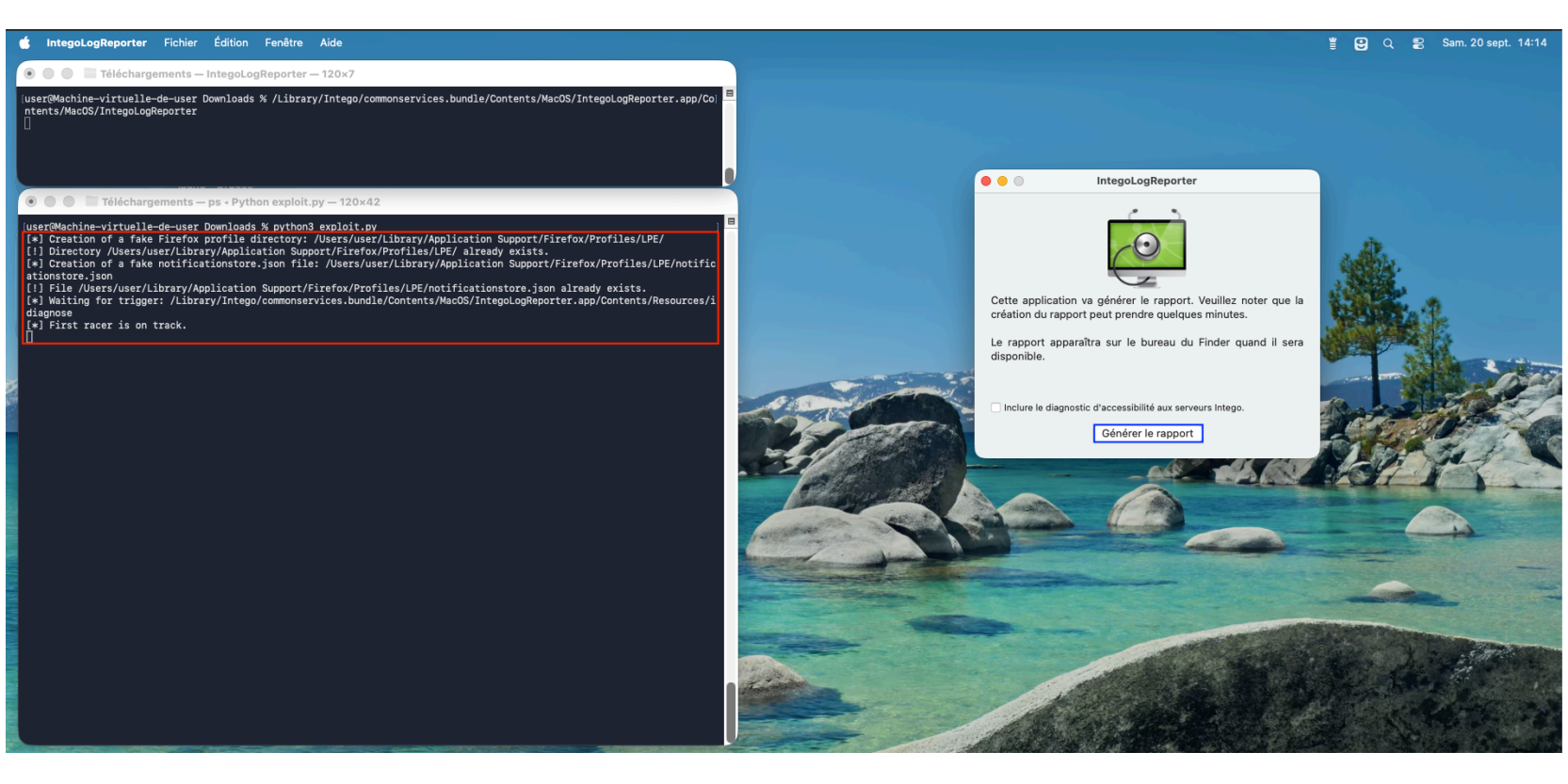


Figure 3 - Execution of the exploit (part 1).

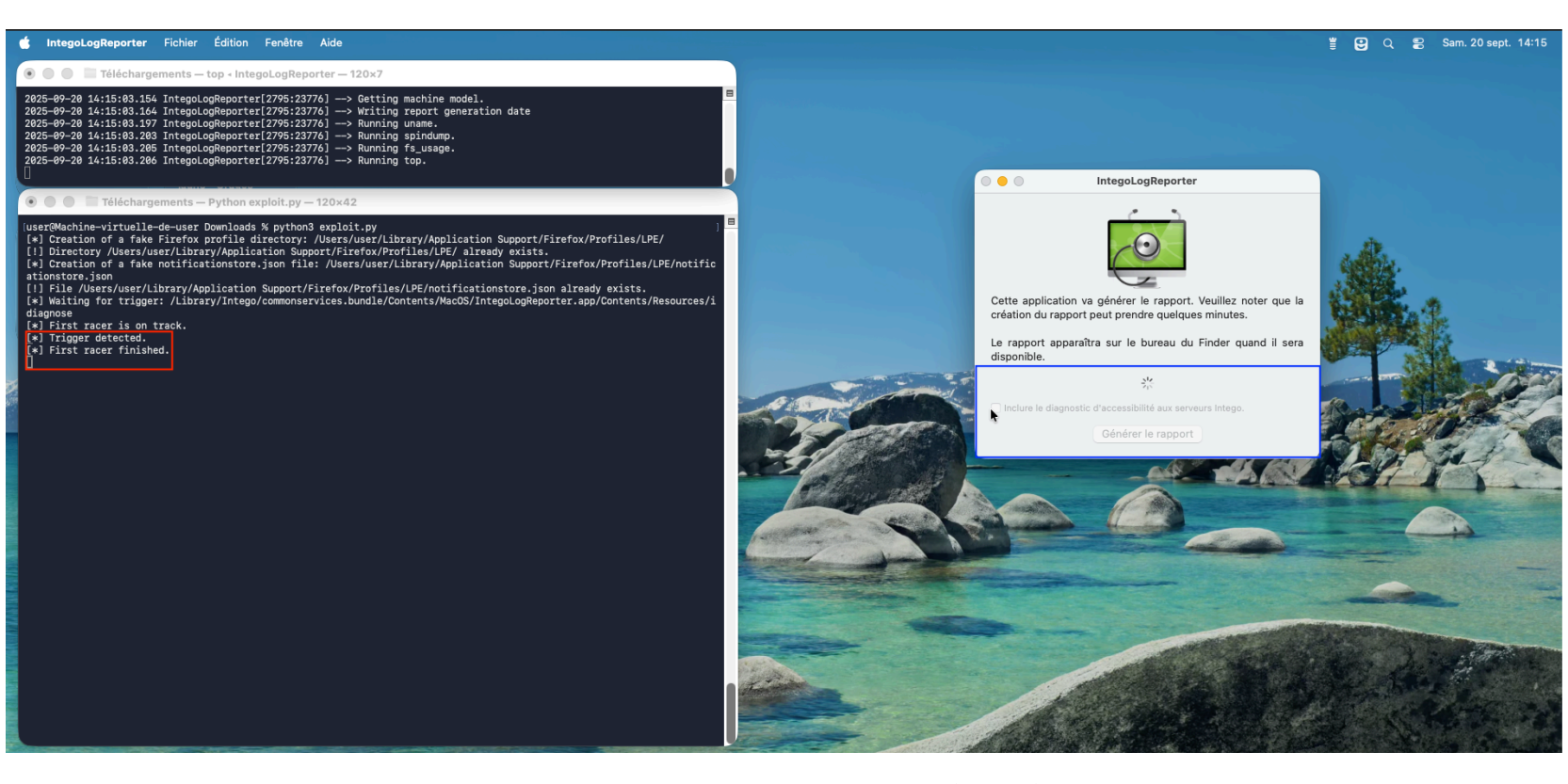


Figure 4 - Execution of the exploit (part 2).

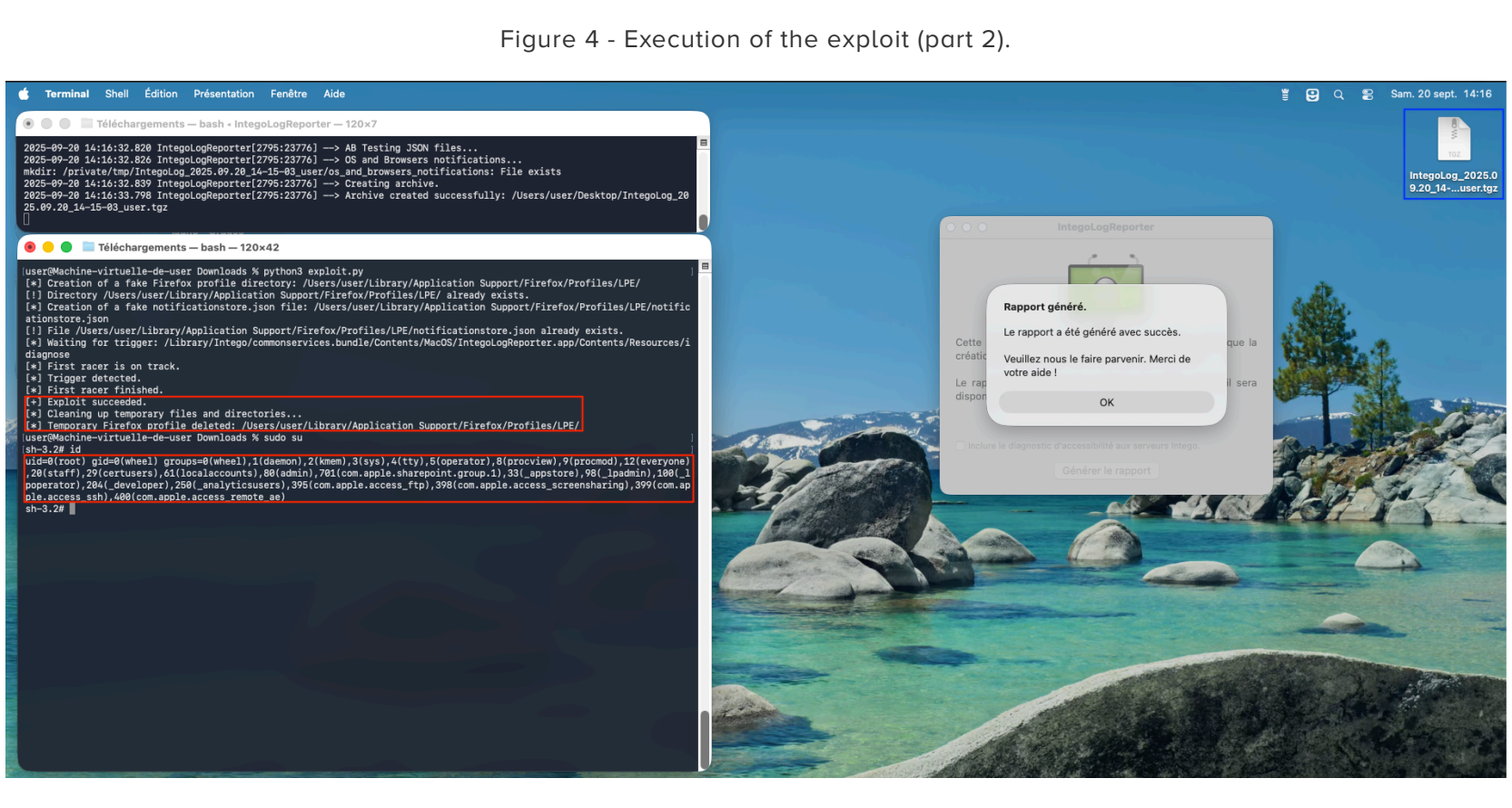
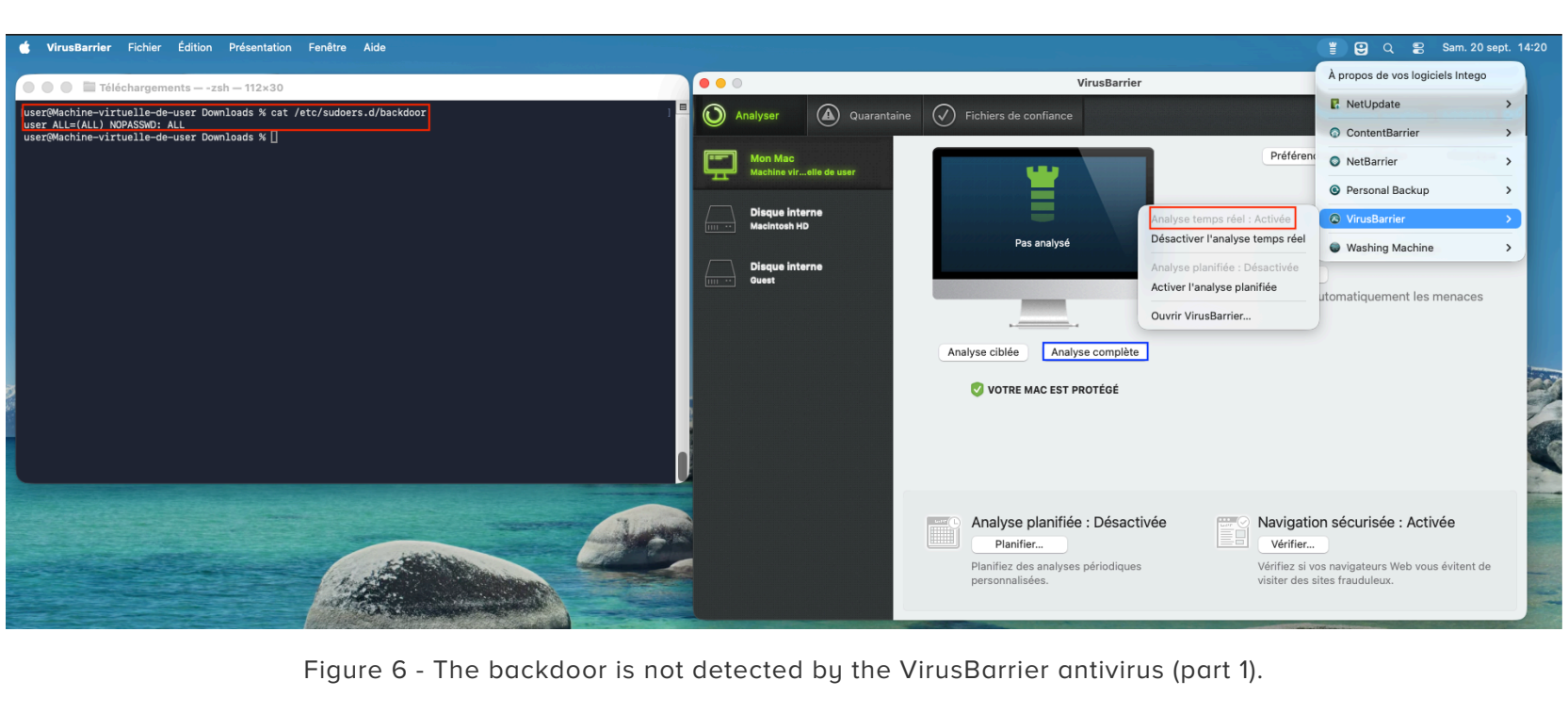


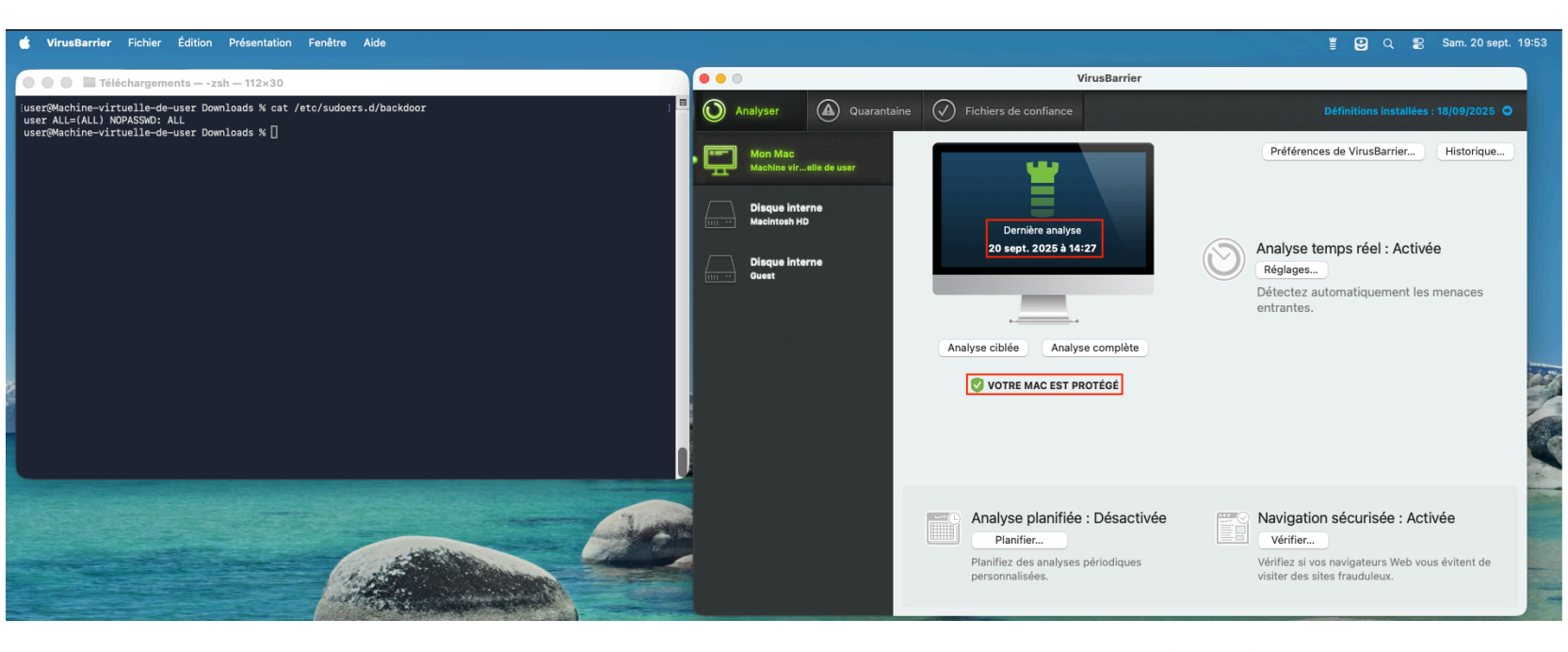
Figure 5 - Execution of the exploit (part 3).

It is interesting to note that once a malware scan is launched, Intego VirusBarrier does not detect the backdoor in /etc/sudoers.d/.



```
user@Machine-virtuelle-de-user Downloads % cat /etc/sudoers.d/backdoor
user ALL=(ALL) NOPASSWD: ALL
user@Machine-virtuelle-de-user Downloads %
```

Figure 6 - The backdoor is not detected by the VirusBarrier antivirus (part 1).



```
user@Machine-virtuelle-de-user Downloads % cat /etc/sudoers.d/backdoor
user ALL=(ALL) NOPASSWD: ALL
user@Machine-virtuelle-de-user Downloads %
```

VirusBarrier
Dernière analyse
20 sept. 2025 à 14:27
VOTRE MAC EST PROTÉGÉ
Analyse temps réel : Activée
Analyse planifiée : Désactivée
Navigation sécurisée : Activée

Figure 7 - The backdoor is not detected by the VirusBarrier antivirus (part 2).

Intego Personal Backup Local Privilege Escalation (as `root`) - CVE-2026-26225

- CVE identifier:
  - CVE-2026-26225

Intego Personal Backup is a macOS utility designed to perform backup operations, allowing user data, applications, and system files to be copied to external drives, mounted network volumes, or other storage locations.

The backups can be scheduled or executed manually, and synchronization between folders or devices is supported. Additionally, bootable clones of a system volume can be created, enabling system recovery through external media. The application uses incremental backup methods to reduce the amount of data transferred after the initial operation, minimizing redundancy and optimizing performance.

We were interested in how information relating to a backup task was stored. First, we created a task.

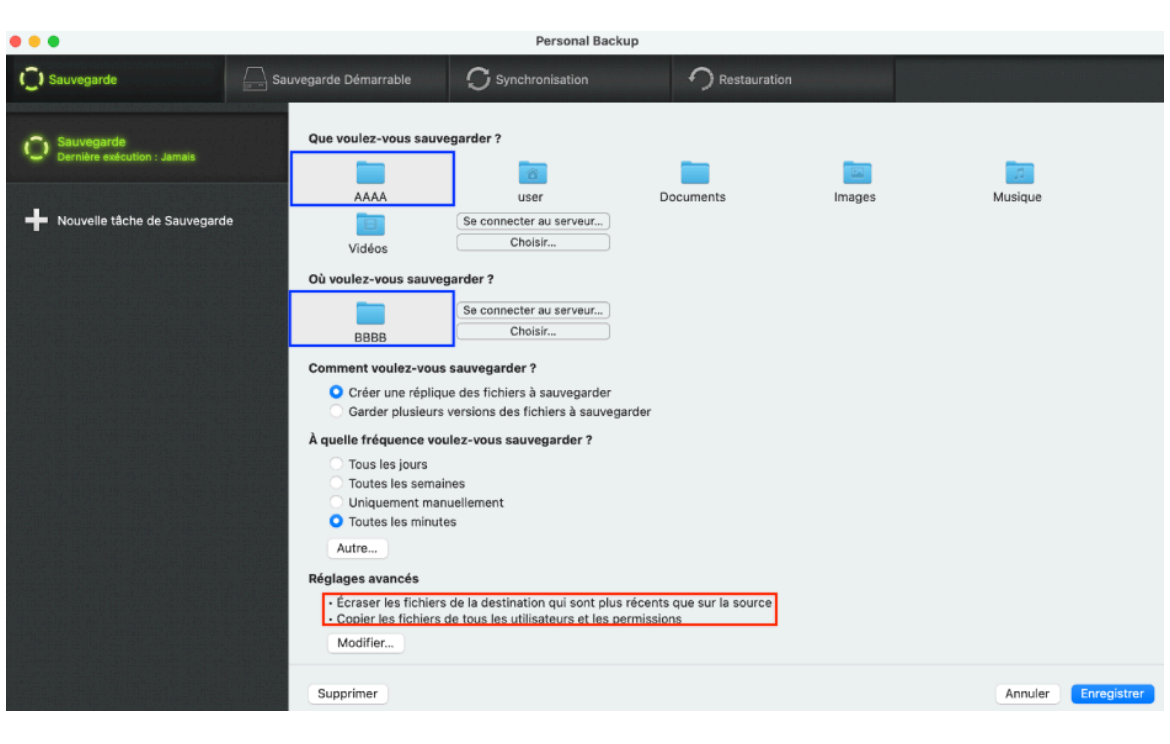

Personal Backup
Sauvegarde
Sauvegarde Démarrable
Synchronisation
Restauration
Nouvelle tâche de Sauvegarde
Que voulez-vous sauvegarder ?
AAAA
user
Documents
Images
Musique
Vidéos
Où voulez-vous sauvegarder ?
BBBB
Comment voulez-vous sauvegarder ?
Créer une réplique des fichiers à sauvegarder
À quelle fréquence voulez-vous sauvegarder ?
Toutes les minutes
Réglages avancés
Écraser les fichiers de la destination qui sont plus récents que sur la source
Copier les fichiers de tous les utilisateurs et les permissions
Supprimer
Annuler
Enregistrer

Figure 8 - Creation of a backup task named "Sauvegarde".

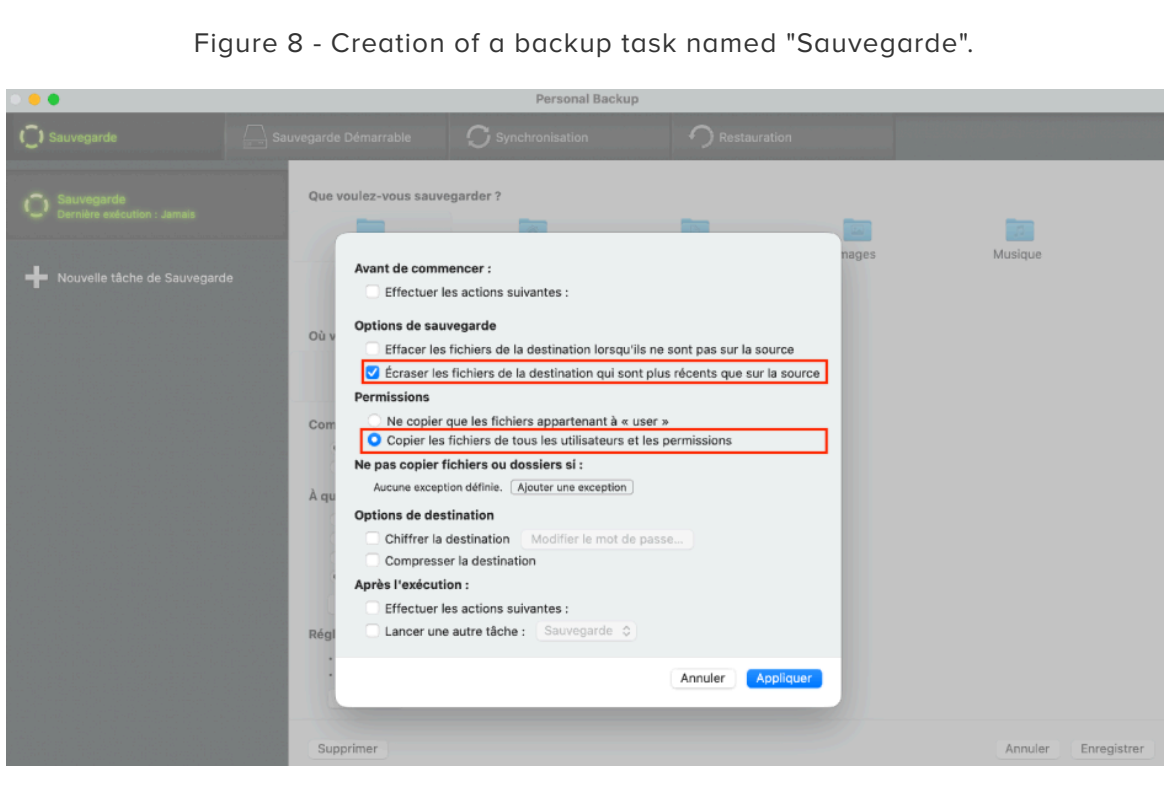
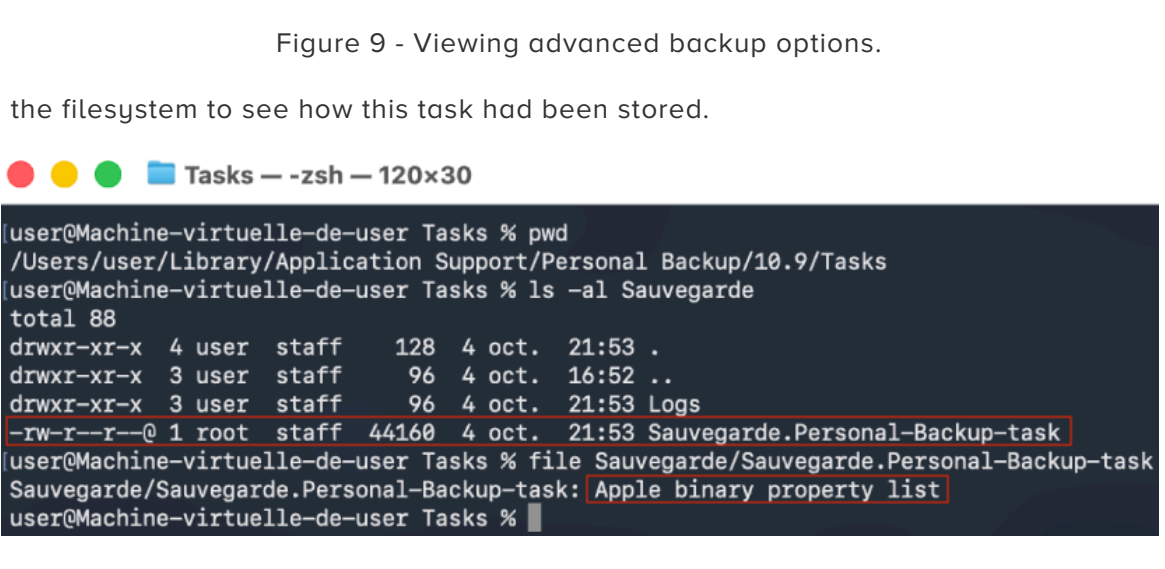

Options de sauvegarde
☑ Écraser les fichiers de la destination qui sont plus récents que sur la source
Permissions
● Copier les fichiers de tous les utilisateurs et les permissions
Annuler
Appliquer

Figure 9 - Viewing advanced backup options.

Then, we explored the filesystem to see how this task had been stored.



```
user@Machine-virtuelle-de-user Tasks % pwd
/Users/user/Library/Application Support/Personal Backup/10.9/Tasks
user@Machine-virtuelle-de-user Tasks % ls -al Sauvegarde
total 88
drwxr-xr-x  4 user  staff    128  4 oct.  21:53 .
drwxr-xr-x  3 user  staff     96  4 oct.  16:52 ..
drwxr-xr-x  3 user  staff     96  4 oct.  21:53 Logs
-rw-r--r--@ 1 root  staff  44160  4 oct.  21:53 Sauvegarde.Personal-Backup-task
user@Machine-virtuelle-de-user Tasks % file Sauvegarde/Sauvegarde.Personal-Backup-task
Sauvegarde/Sauvegarde.Personal-Backup-task: Apple binary property list
user@Machine-virtuelle-de-user Tasks %
```

Figure 10 - Storage of the task on the file system in the form of a binary .plist file.

Since the directory is accessible for reading and writing as our non-privileged user (but the task file is owned by `root`), we figured that if we could understand what the .plist file contained, we would be able to manually craft our own task.

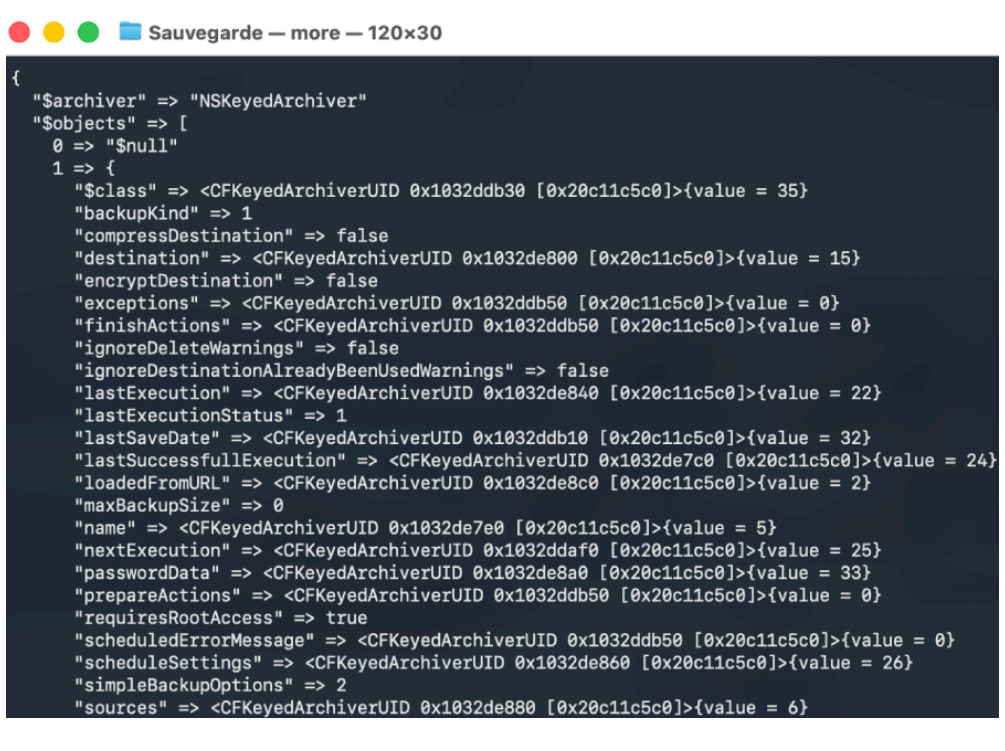
Analysis of the task file (.plist)

We used `plutil` to analyze the task file.

Command:

```
plutil -p Sauvegarde.Personal-Backup-task | more
```

Output:



```
{
  "$archiver" => "NSKeyedArchiver"
  "$objects" => [
    0 => "$null"
    1 => {
      "$class" => <CFKeyedArchiverUID 0x1032ddb30 [0x20c11c5c0]>{value = 35}
      "backupKind" => 1
      "compressDestination" => false
      "destination" => <CFKeyedArchiverUID 0x1032de800 [0x20c11c5c0]>{value = 15}
      "encryptDestination" => false
      "exceptions" => <CFKeyedArchiverUID 0x1032ddb50 [0x20c11c5c0]>{value = 0}
      "finishActions" => <CFKeyedArchiverUID 0x1032ddb50 [0x20c11c5c0]>{value = 0}
      "ignoreDeleteWarnings" => false
      "ignoreDestinationAlreadyBeenUsedWarnings" => false
      "lastExecution" => <CFKeyedArchiverUID 0x1032de840 [0x20c11c5c0]>{value = 22}
      "lastExecutionStatus" => 1
      "lastSaveDate" => <CFKeyedArchiverUID 0x1032ddb10 [0x20c11c5c0]>{value = 32}
      "lastSuccessfullExecution" => <CFKeyedArchiverUID 0x1032de7c0 [0x20c11c5c0]>{value = 24}
      "loadedFromURL" => <CFKeyedArchiverUID 0x1032de8c0 [0x20c11c5c0]>{value = 2}
      "maxBackupSize" => 0
      "name" => <CFKeyedArchiverUID 0x1032de7e0 [0x20c11c5c0]>{value = 5}
      "nextExecution" => <CFKeyedArchiverUID 0x1032ddaf0 [0x20c11c5c0]>{value = 25}
      "passwordData" => <CFKeyedArchiverUID 0x1032de8a0 [0x20c11c5c0]>{value = 33}
      "prepareActions" => <CFKeyedArchiverUID 0x1032ddb50 [0x20c11c5c0]>{value = 0}
      "requiresRootAccess" => true
      "scheduledErrorMessage" => <CFKeyedArchiverUID 0x1032ddb50 [0x20c11c5c0]>{value = 0}
      "scheduleSettings" => <CFKeyedArchiverUID 0x1032de860 [0x20c11c5c0]>{value = 26}
      "simpleBackupOptions" => 2
      "sources" => <CFKeyedArchiverUID 0x1032de880 [0x20c11c5c0]>{value = 6}
```

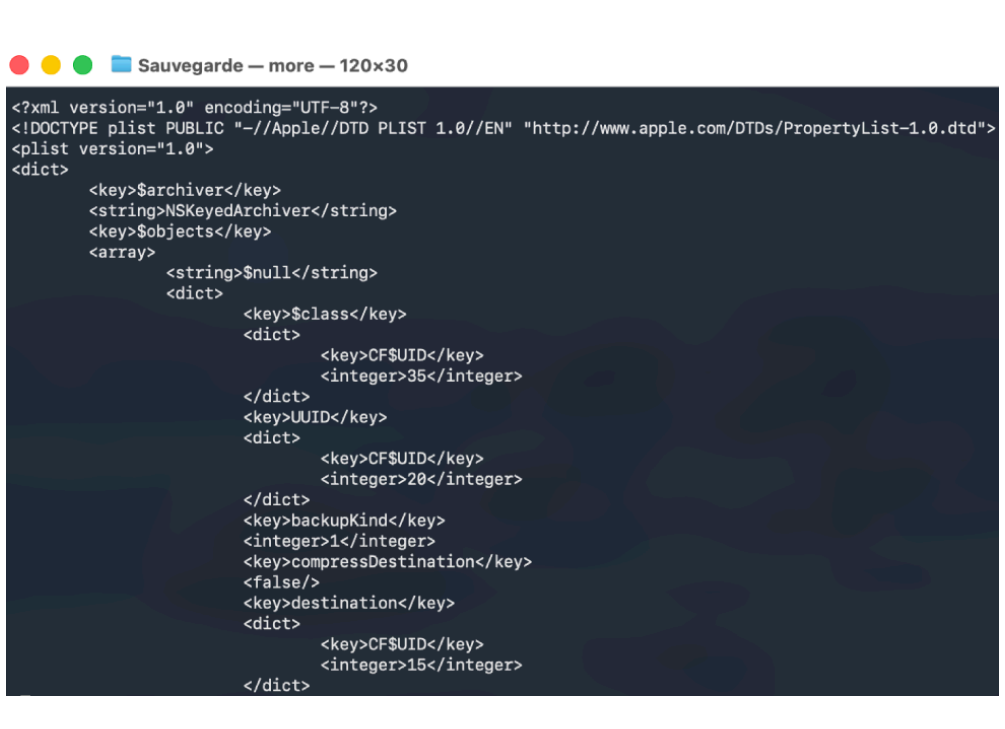
We concluded that this file was serialized, as it describes the full setup of our task called "Sauvegarde" (including information such as the source and destination folders, the backup schedule, etc.) and makes direct references to `NSKeyedArchiver`.

To gain a clearer understanding we converted the binary format back into XML using `plutil`.

Command:

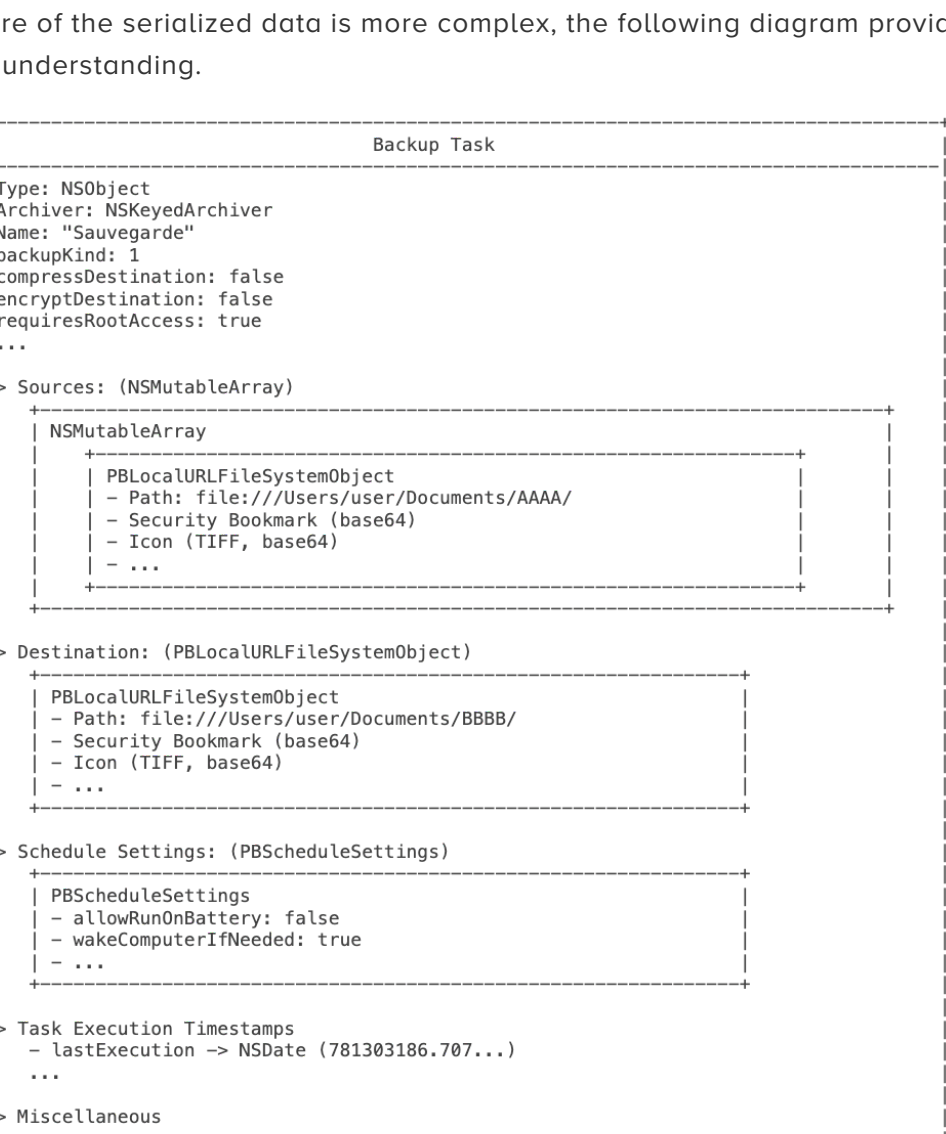
```
plutil -convert xml1 -o - Sauvegarde.Personal-Backup-task | more
```

Output:



```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
        <key>$archiver</key>
        <string>NSKeyedArchiver</string>
        <key>$objects</key>
        <array>
                <string>$null</string>
                <dict>
                        <key>$class</key>
                        <dict>
                                <key>CF$UID</key>
                                <integer>35</integer>
                        </dict>
                        <key>UUID</key>
                        <dict>
                                <key>CF$UID</key>
                                <integer>20</integer>
                        </dict>
                        <key>backupKind</key>
                        <integer>1</integer>
                        <key>compressDestination</key>
                        <false/>
                        <key>destination</key>
                        <dict>
                                <key>CF$UID</key>
                                <integer>15</integer>
                        </dict>
```

Although the actual structure of the serialized data is more complex, the following diagram provides a simplified representation to facilitate understanding.



```
+--------------------------------------------------------------------------------------+
|                                     Backup Task                                      |
|--------------------------------------------------------------------------------------|
| Type: NSObject                                                                       |
| Archiver: NSKeyedArchiver                                                            |
| Name: "Sauvegarde"                                                                   |
| backupKind: 1                                                                        |
| compressDestination: false                                                           |
| encryptDestination: false                                                            |
| requiresRootAccess: true                                                             |
| ...                                                                                  |
|                                                                                      |
| > Sources: (NSMutableArray)                                                          |
|    +---------------------------------------------------------------------------+     |
|    | NSMutableArray                                                            |     |
|    |    +---------------------------------------------------------------+      |     |
|    |    | PBLocalURLFileSystemObject                                    |      |     |
|    |    | - Path: file:///Users/user/Documents/AAAA/                    |      |     |
|    |    | - Security Bookmark (base64)                                  |      |     |
|    |    | - Icon (TIFF, base64)                                         |      |     |
|    |    | - ...                                                         |      |     |
|    |    +---------------------------------------------------------------+      |     |
|    +---------------------------------------------------------------------------+     |
|                                                                                      |
| > Destination: (PBLocalURLFileSystemObject)                                          |
|    +----------------------------------------------------------------+                |
|    | PBLocalURLFileSystemObject                                     |                |
|    | - Path: file:///Users/user/Documents/BBBB/                     |                |
|    | - Security Bookmark (base64)                                   |                |
|    | - Icon (TIFF, base64)                                          |                |
|    | - ...                                                          |                |
|    +----------------------------------------------------------------+                |
|                                                                                      |
| > Schedule Settings: (PBScheduleSettings)                                            |
|    +----------------------------------------------------------------+                |
|    | PBScheduleSettings                                             |                |
|    | - allowRunOnBattery: false                                     |                |
|    | - wakeComputerIfNeeded: true                                   |                |
|    | - ...                                                          |                |
|    +----------------------------------------------------------------+                |
|                                                                                      |
| > Task Execution Timestamps                                                          |
|    - lastExecution -> NSDate (781303186.707...)                                      |
|    ...                                                                               |
|                                                                                      |
| > Miscellaneous                                                                      |
|    ...                                                                               |
+--------------------------------------------------------------------------------------+
```

Figure 11 - Simplified diagram of serialized data.

We created a second backup task and performed a comparison (`diff`) with the original to identify any data changes. To make the differences easier to spot, we deliberately modified the source and destination paths. Our analysis indicates that, aside from the expected changes in timestamps, the most significant difference were in the values of the Apple security bookmarks.

> ℹ️ Apple security bookmarks are a mechanism that allows macOS to securely store and later access files or directories.

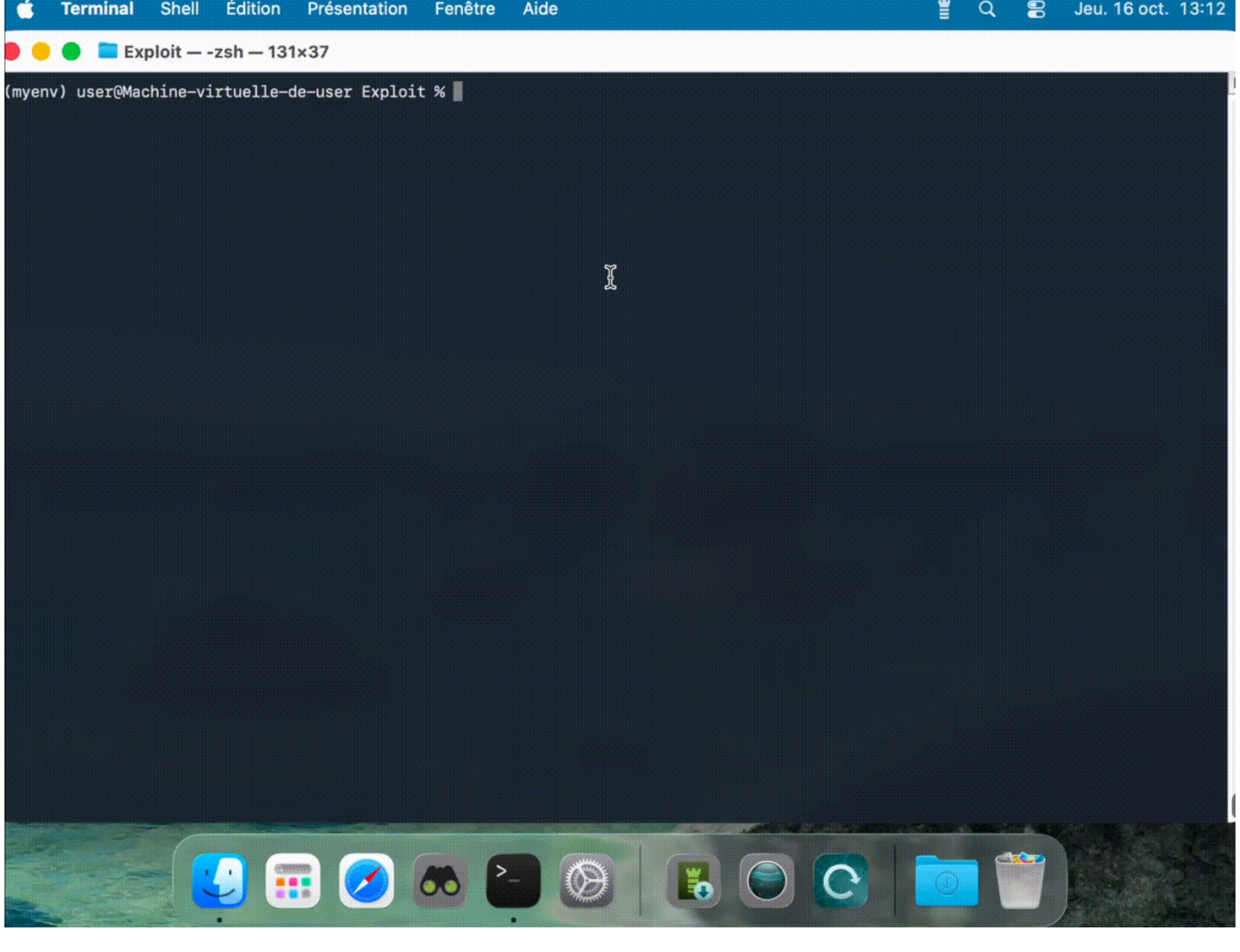
To achieve the Local Privilege Escalation (LPE), we created an XML .plist template that represents a task (including placeholders for TIFF icons data and security bookmarks). The exploit generates a valid .plist from that template, places it in the expected directory.

> The reason why the exploit overwrites /etc/pam.d/su is explained in the next section.

Exploit

The exploit is packaged as follows:

- Exploit/
  - FakePamFolder/
    - su
  - RUN.txt
  - exploit.py
  - folder_icon.tiff
  - template.plist
  - user-preferences.plist



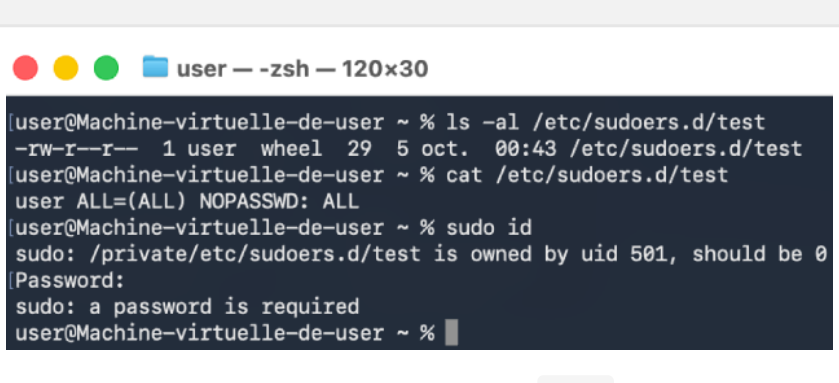
```
(myenv) user@Machine-virtuelle-de-user Exploit %
```

Figure 12 - Exploit running.

Why did we decide to write to /etc/pam.d/su and not /etc/sudoers.d/?

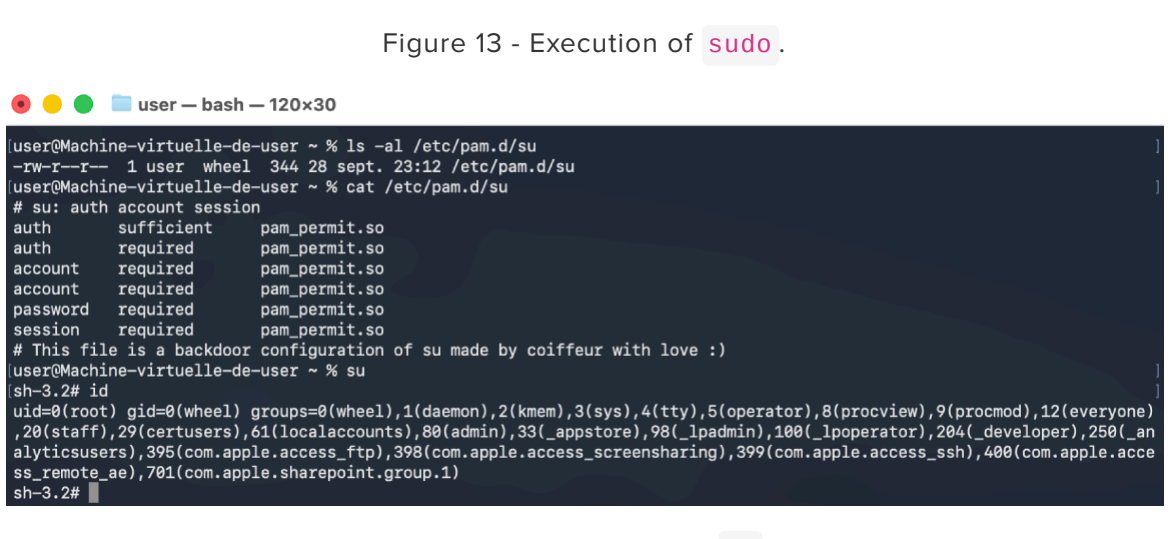
Although the exploit allows writing a file that is initially owned by `root`, the ownership is changed to `user:wheel` after execution. The PAM (Pluggable Authentication Modules) system does not seem to enforce strict ownership checks on its configuration files, so a configuration file owned by a standard user will still be considered. By contrast, `sudo` enforces ownership and will ignore files that are not properly owned by `root`. In short, a user-owned PAM file can be applied by the system, whereas a user-owned sudo file will be ignored, which is an important distinction when choosing where to place malicious configuration changes.

> Test performed on the latest version of macOS (macOS 26.0.1 Tahoe).



```
user@Machine-virtuelle-de-user ~ % ls -al /etc/sudoers.d/test
-rw-r--r--  1 user  wheel  29  5 oct.  00:43 /etc/sudoers.d/test
user@Machine-virtuelle-de-user ~ % cat /etc/sudoers.d/test
user ALL=(ALL) NOPASSWD: ALL
user@Machine-virtuelle-de-user ~ % sudo id
sudo: /private/etc/sudoers.d/test is owned by uid 501, should be 0
Password:
sudo: a password is required
user@Machine-virtuelle-de-user ~ %
```

Figure 13 - Execution of `sudo`.



```
user@Machine-virtuelle-de-user ~ % ls -al /etc/pam.d/su
-rw-r--r--  1 user  wheel  344 28 sept. 23:12 /etc/pam.d/su
user@Machine-virtuelle-de-user ~ % cat /etc/pam.d/su
# su: auth account session
auth       sufficient     pam_permit.so
auth       required       pam_permit.so
account    required       pam_permit.so
account    required       pam_permit.so
password   required       pam_permit.so
session    required       pam_permit.so
# This file is a backdoor configuration of su made by coiffeur with love :)
user@Machine-virtuelle-de-user ~ % su
sh-3.2# id
uid=0(root) gid=0(wheel) groups=0(wheel),1(daemon),2(kmem),3(sys),4(tty),5(operator),8(procview),9(procmod),12(everyone)
,20(staff),29(certusers),61(localaccounts),80(admin),33(_appstore),98(_lpadmin),100(_lpoperator),204(_developer),250(_an
alyticsusers),395(com.apple.access_ftp),398(com.apple.access_screensharing),399(com.apple.access_ssh),400(com.apple.acce
ss_remote_ae),701(com.apple.sharepoint.group.1)
sh-3.2#
```

Figure 14 - Execution of `su`.

Conclusion

In this post, we explored two local privilege escalation (LPE) vulnerabilities in Intego on macOS, both stemming from TOCTOU (time-of-check to time-of-use) issues. These examples highlight how seemingly minor race conditions can lead to significant security risks. In the next post, we will dive into how reverse engineering Intego's XPC services uncovered additional attack vectors, revealing even more potential avenues for LPE. Stay tuned to see how a deeper understanding of these services can shed light on further vulnerabilities.

Disclosure Timeline

Below we include a timeline of all the relevant events during the coordinated vulnerability disclosure process with the intent of providing transparency to the whole process and our actions.

- 2025-11-06: Quarkslab sent mail to dpo@intego.com and asked for a security point of contact to report vulnerabilities.
- 2025-12-08: Quarkslab sent mail to info@intego.com and asked for a security point of contact to report vulnerabilities.
- 2025-12-16: Quarkslab sent the vulnerability report to CERT-FR and indicated it had not been able to contact the vendor and that the disclosure date was set to December 30th, 2025.
- 2025-12-17: CERT-FR acknowledged the report and asked which contacts did Quarkslab try. Suggested to postpone the publication until mid-February to give them time to attempt coordination with the vendor and to avoid publishing at the end of the year.
- 2025-12-18: Quarkslab agreed to postpone publication to February 10th, 2026 and provided the emails of attempted contact
- 2025-12-24: CERT-FR asked which exact versions were tested and asked if they could send the report to the vendor.
- 2025-12-24: CERT-FR contacted the vendor via its support point of contact.
- 2026-01-15: CERT-FR contacted the vendor and reminded them that publication was planned for February 10th. Asked for plans to release fixes.
- 2026-01-17: Intego customer support replied the report had already been forwarded to the appropriate department for review, and that they would provide an update via email as soon as more information becomes available.
- 2026-01-24: CERT-FR informed Quarkslab of the ongoing disclosure coordination and said that they indicated them that in the absence of detailed feedback regarding the handling of the report, publication would proceed as agreed in February.
- 2026-02-05: Quarkslab sent mail to CERT-FR saying the publication will proceed as agreed.
- 2026-02-10: This blog post is published.
- 2026-02-12: CVE-2026-26224 & CVE-2026-26225 made official for referencing vulnerabilities.

References

- Sniffing Authentication References on macOS, Patrick Wardle

---

If you would like to learn more about our security audits and explore how we can help you, get in touch with us!